HW5 —Tutte Parameterization 实验报告

胡广理 PB24010500

2026 年 4 月 13 日

摘要

本实验实现并比较了基于均匀权重（Uniform）与余切权重（Cotangent）的 Tutte 参数化，并在三维模型表面回写 UV 后贴图展示结果。本文记录了完整的操作步骤、节点连线、关键参数值、实验结果和文件输出位置（含 'Results.usd'）。

1 环境与文件位置

- 项目根目录：USTC_CG（略）- 实验图像：`Homeworks/5_tutte_parametrzation/pdf/pictures/` - 输入模型：`Homeworks/5_tutte_parametrzation/data/meshes/Balls.usda` - 贴图图像：`Homeworks/5_tutte_parametrzation/data/test.png` - 本次最终保存的 USD：`Homeworks/5_tutte_parametrzation/pdf/Results.usd`（注意：你打开时使用的是本机的绝对路径，保存时请确认该文件路径对提交包有效）

2 节点与数据流概览

本实验在 Ruzino 的节点编辑器按照下列数据流执行：

1. `read_usd`（读取 USD 模型）
2. `hw5_boundary_map`（可选：将边界映射到凸平面区域—方形或圆形）
3. `hw5_param`（求解 Tutte 参数化，输出参数化几何并写入顶点 `Texcoords`）
4. `mesh_decompose`（从原始立体读出顶点/拓扑）
5. `mesh_decompose`（从参数化结果读出 `Texcoords`）
6. `mesh_compose`（将原始顶点/拓扑与参数化得到的 `Texcoords` 组合回一个带 UV 的 Mesh）
7. `set_texture`（将 `test.png` 赋给该 Mesh 的材质）
8. `write_usd`（写入 stage）

3 详细操作步骤（逐步精确）

下面列出在 Ruzino GUI 中逐步操作（点击、填值、拖拽均写明）：

3.1 读取模型

1. 右键 → 搜索 read_usd → 添加节点。2. 在参数面板：File Name 填入：/home/hgl/code/ustc/USTC_CG/Homeworks/5_tutte_parametrzation/data/meshes/Balls.usda。Prim Path 填入：/Balls/Balls。Time Code=0。

3.2 参数化（生成 UV）

1.（可选）添加 hw5_boundary_map，将 read_usd 输出拖到它的 Geometry 输入；在 Shape 填 square 或 circle（分别得到方形/圆形边界映射）。2. 添加 hw5_param，将 hw5_boundary_map 的输出（或直接 read_usd 输出）连到 hw5_param 的 Geometry。3. 若使用余切权重，额外将 read_usd 输出连到 hw5_param 的 Reference Geometry 输入。4. 在 hw5_param 的 Weight Type 填入：uniform 或 cotangent（节点内部会归一化并转为小写）。

3.3 提取与重构用于贴图的立体网格

1. 添加第一个 mesh_decompose，连接 read_usd 输出到其 Mesh 输入。该节点输出 Vertices、FaceVertexCounts、FaceVertexIndices、Normals。2. 添加第二个 mesh_decompose，连接 hw5_param 输出到其 Mesh 输入，取其 Texcoords 输出（参数化得到的 UV）。3. 添加 mesh_compose，并将第一个 mesh_decompose 的 Vertices、FaceVertexCounts、FaceVertexIndices、Normals 分别连接到 mesh_compose 对应输入，将第二个 mesh_decompose 的 Texcoords 连接到 mesh_compose 的 Texcoords。

3.4 贴图与导出

1. 将 mesh_compose 的输出连到 set_texture 的 Geometry，在 set_texture 的 Texture Name 输入：/home/hgl/code/ustc/USTC_CG/Homeworks/5_tutte_parametrzation/data/test.png（绝对路径以避免解析为相对于可执行程序的相对路径错误）。2. 将 set_texture 输出连到 write_usd 的 Geometry 输入，在 write_usd 的 Sub Path 填 /Result（或其他 prim 名）。3. 在 Ruzino 主菜单执行：File → Save Stage As...，填写保存路径（例如 Homeworks/5_tutte_parametrzation/pd
确认保存。

4 实验结果（图示）

下面插入你放在 pdf/pictures 下的图片，按需要在报告中替换或裁剪。

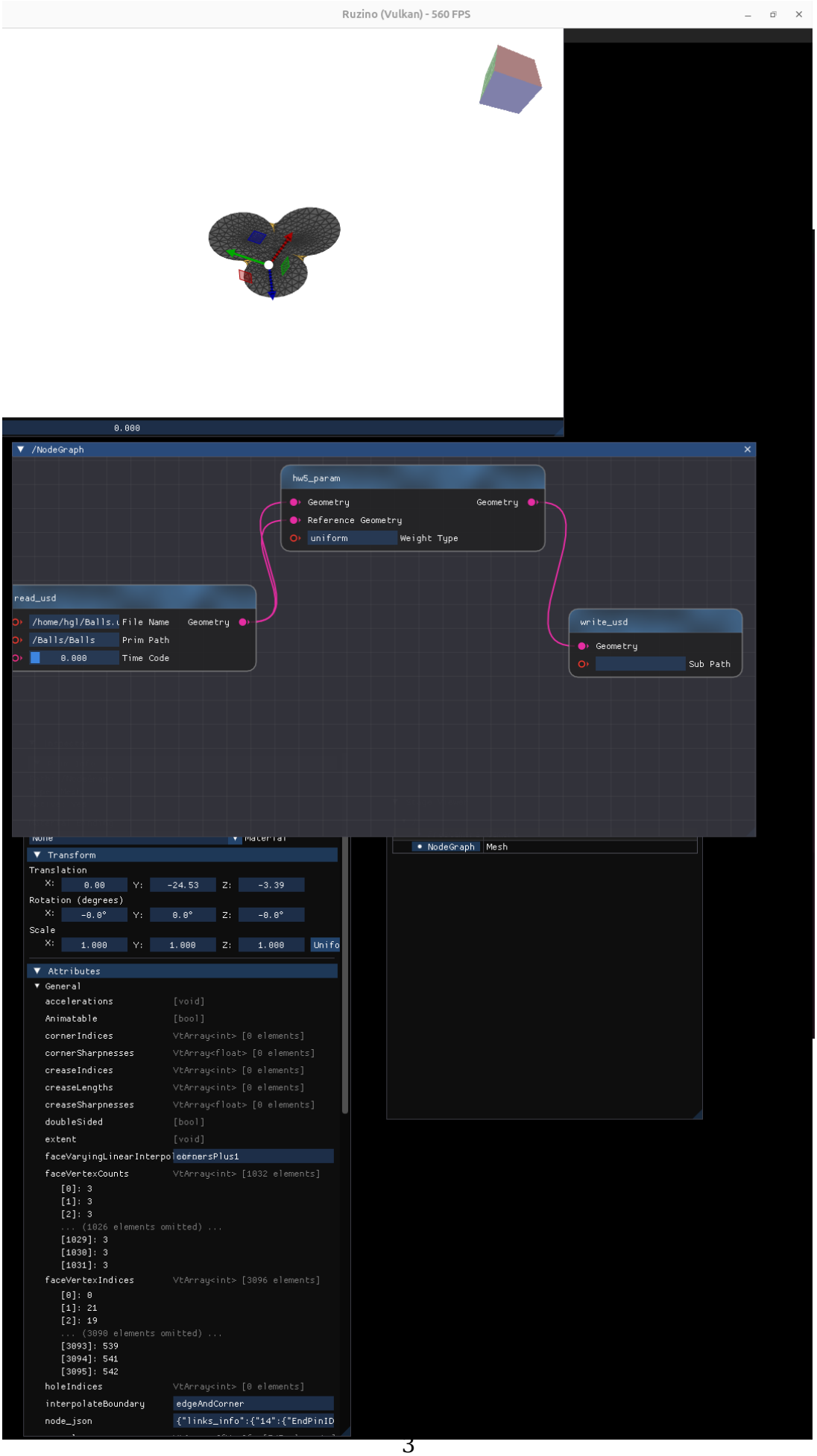

图 1：均匀权重（Uniform）初始参数化（平面）

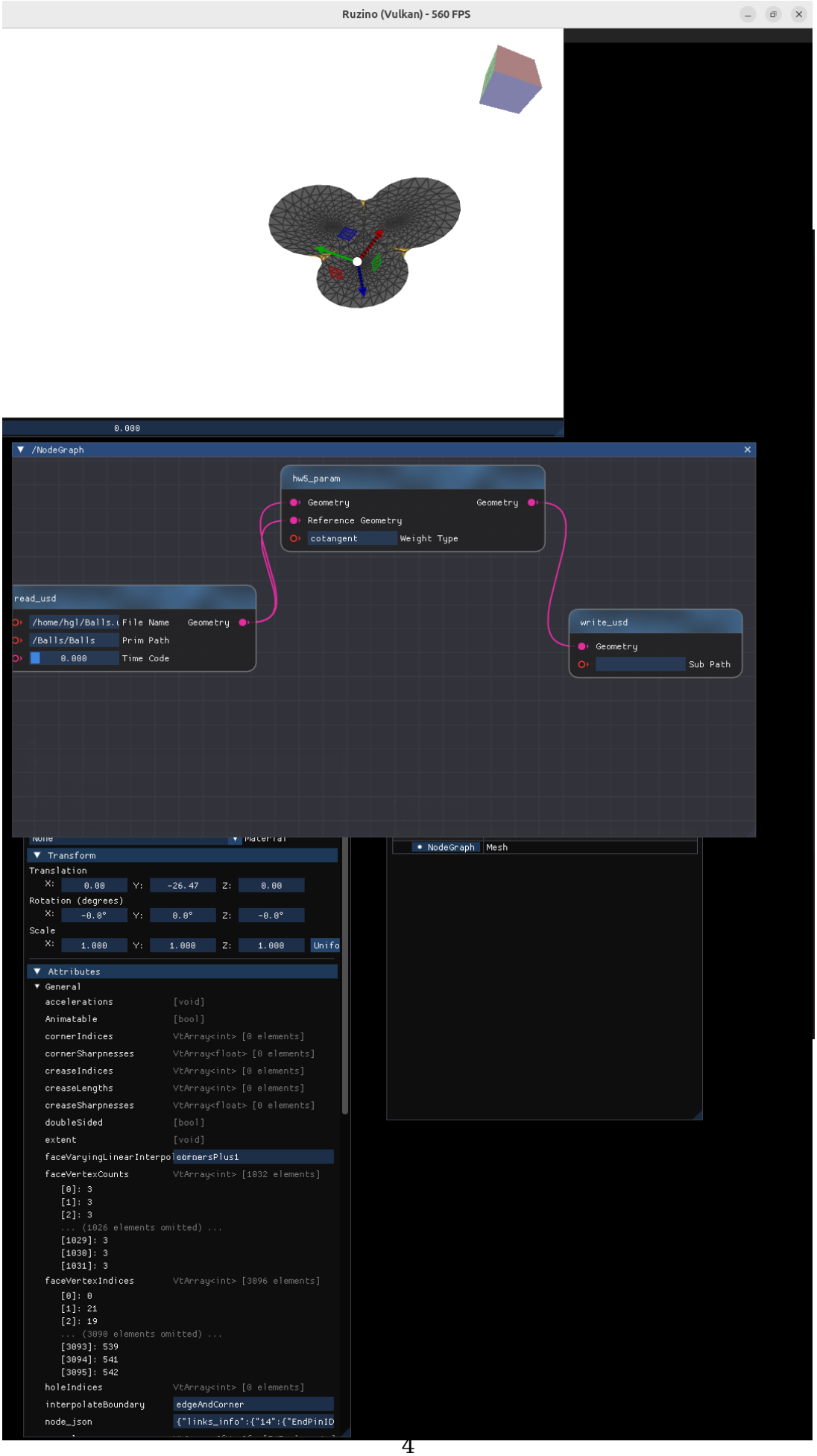

图 2: 余切权重（Cotangent）初始参数化（平面）

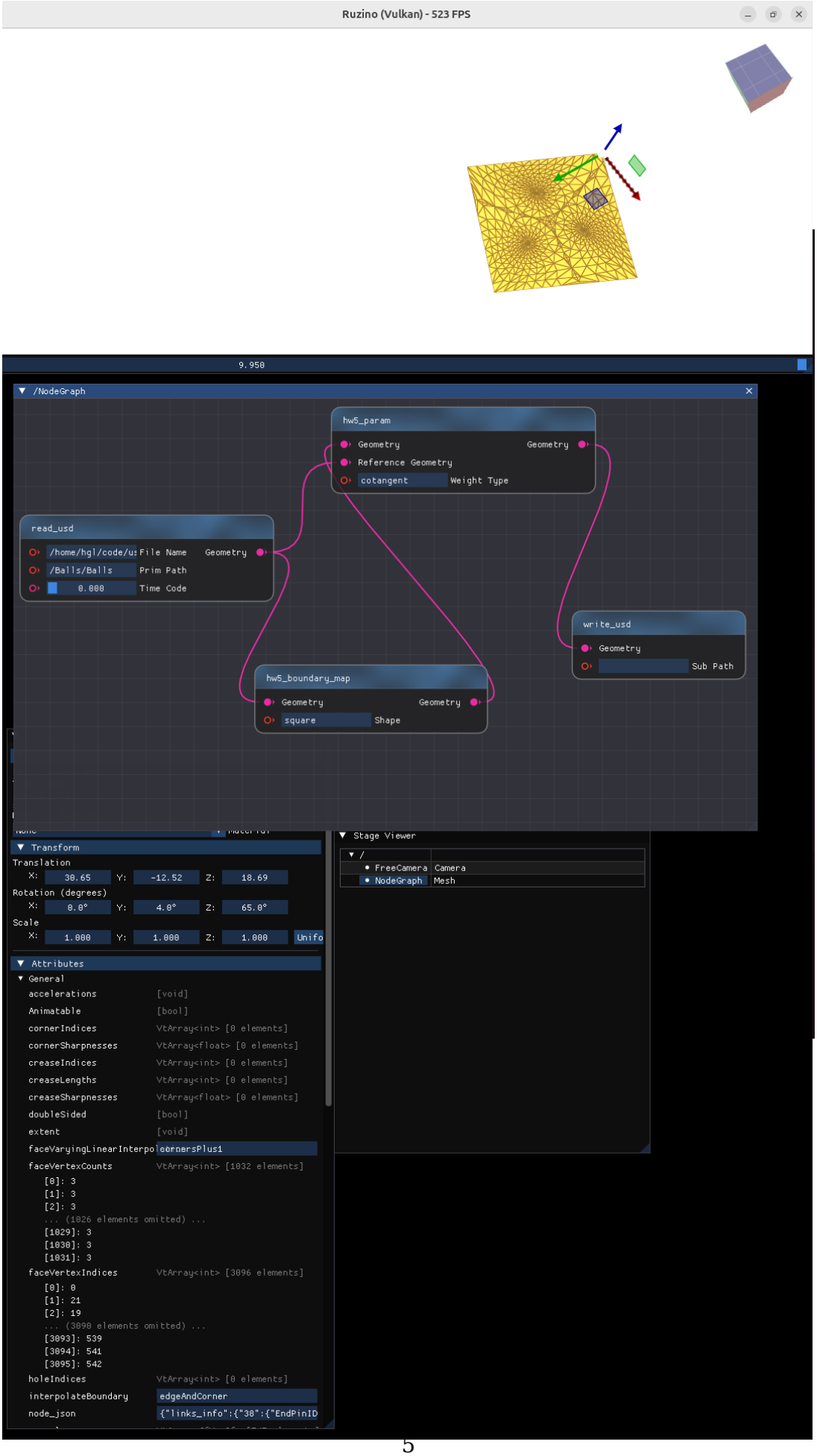
Ruzino (Vulkan) - 523 FPS
9.950
/NodeGraph
hw5_param
Geometry
Geometry
Reference Geometry
cotangent
Weight Type
read_usd
/home/hgl/code/us
File Name
Geometry
/Balls/Balls
Prim Path
0.000
Time Code
write_usd
Geometry
Sub Path
hw5_boundary_map
Geometry
Geometry
square
Shape
Transform
Translation
X: 30.65 Y: -12.52 Z: 18.69
Rotation (degrees)
X: 0.0° Y: 4.0° Z: 65.0°
Scale
X: 1.000 Y: 1.000 Z: 1.000
Attributes
General
accelerations [void]
Animatable [bool]
cornerIndices VtArray<int> [0 elements]
cornerSharpnesses VtArray<float> [0 elements]
creaseIndices VtArray<int> [0 elements]
creaseLengths VtArray<int> [0 elements]
creaseSharpnesses VtArray<float> [0 elements]
doubleSided [bool]
extent [void]
faceVaryingLinearInterpo cornersPlus1
faceVertexCounts VtArray<int> [1032 elements]
[0]: 3
[1]: 3
[2]: 3
... (1026 elements omitted) ...
[1029]: 3
[1030]: 3
[1031]: 3
faceVertexIndices VtArray<int> [3096 elements]
[0]: 0
[1]: 21
[2]: 19
... (3090 elements omitted) ...
[3093]: 539
[3094]: 541
[3095]: 542
holeIndices VtArray<int> [0 elements]
interpolateBoundary edgeAndCorner
node_json {"links_info":{"38":{"EndPinID
Stage Viewer
/
FreeCamera Camera
NodeGraph Mesh

图 3: 边界映射：方形

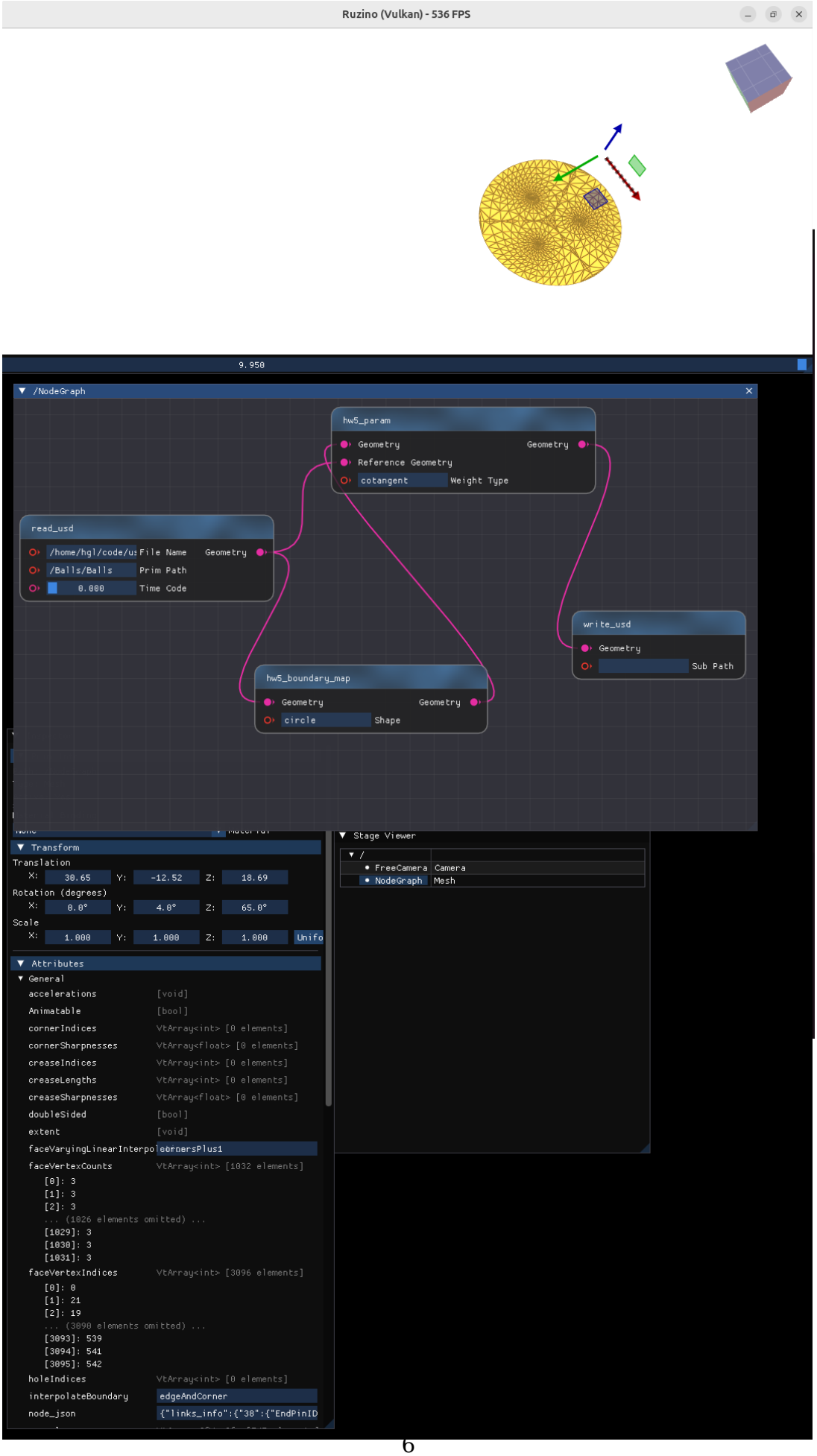
Ruzino (Vulkan) - 536 FPS
9.950
/NodeGraph
hw5_param
Geometry
Geometry
Reference Geometry
cotangent
Weight Type
read_usd
/home/hgl/code/us
File Name
Geometry
/Balls/Balls
Prim Path
0.000
Time Code
write_usd
Geometry
Sub Path
hw5_boundary_map
Geometry
Geometry
circle
Shape
Transform
Translation
X: 30.65 Y: -12.52 Z: 18.69
Rotation (degrees)
X: 0.0° Y: 4.0° Z: 65.0°
Scale
X: 1.000 Y: 1.000 Z: 1.000
Stage Viewer
/
FreeCamera Camera
NodeGraph Mesh
Attributes
General
accelerations [void]
Animatable [bool]
cornerIndices VtArray<int> [0 elements]
cornerSharpnesses VtArray<float> [0 elements]
creaseIndices VtArray<int> [0 elements]
creaseLengths VtArray<int> [0 elements]
creaseSharpnesses VtArray<float> [0 elements]
doubleSided [bool]
extent [void]
faceVaryingLinearInterpo cornersPlus1
faceVertexCounts VtArray<int> [1032 elements]
[0]: 3
[1]: 3
[2]: 3
... (1026 elements omitted) ...
[1029]: 3
[1030]: 3
[1031]: 3
faceVertexIndices VtArray<int> [3096 elements]
[0]: 0
[1]: 21
[2]: 19
... (3090 elements omitted) ...
[3093]: 539
[3094]: 541
[3095]: 542
holeIndices VtArray<int> [0 elements]
interpolateBoundary edgeAndCorner
node_json {"links_info":{"38":{"EndPinID

图 4: 边界映射：圆形

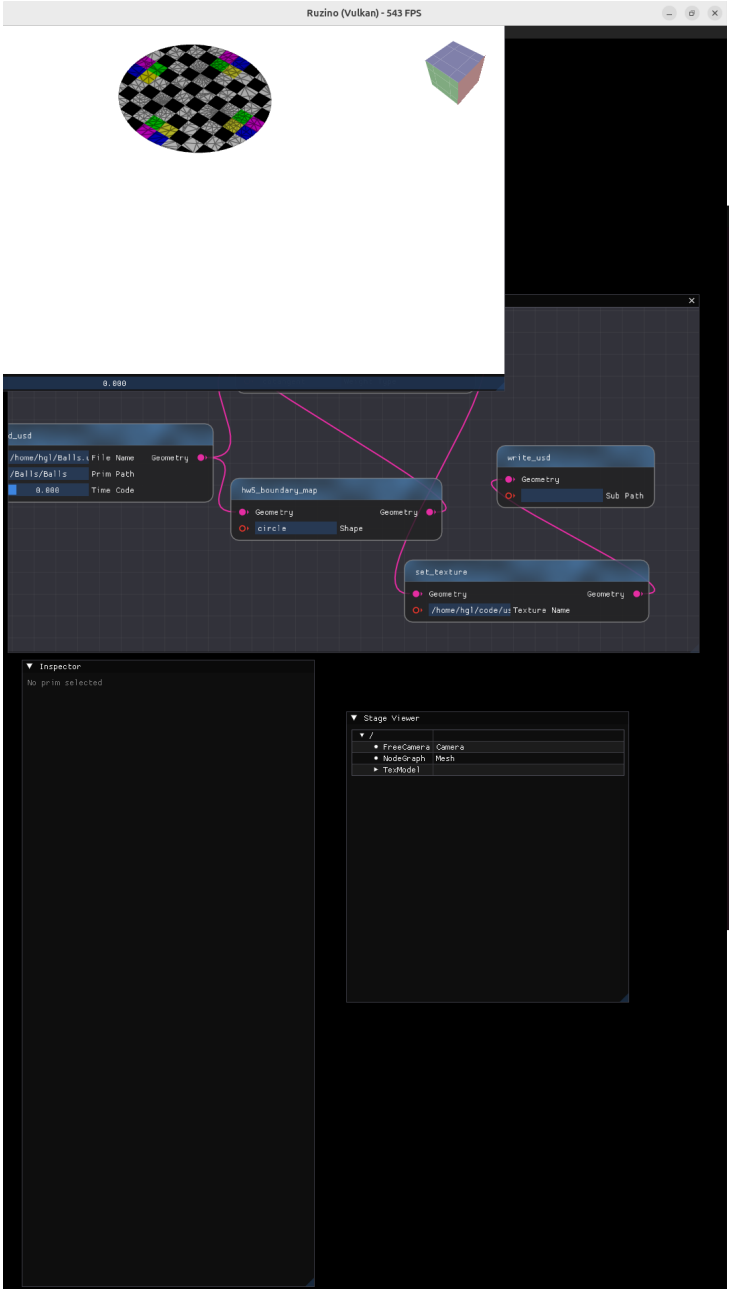

图 5: 将参数化回写至立体并贴上 `test.png` 的效果（2D UV 在立体表面上的展示）

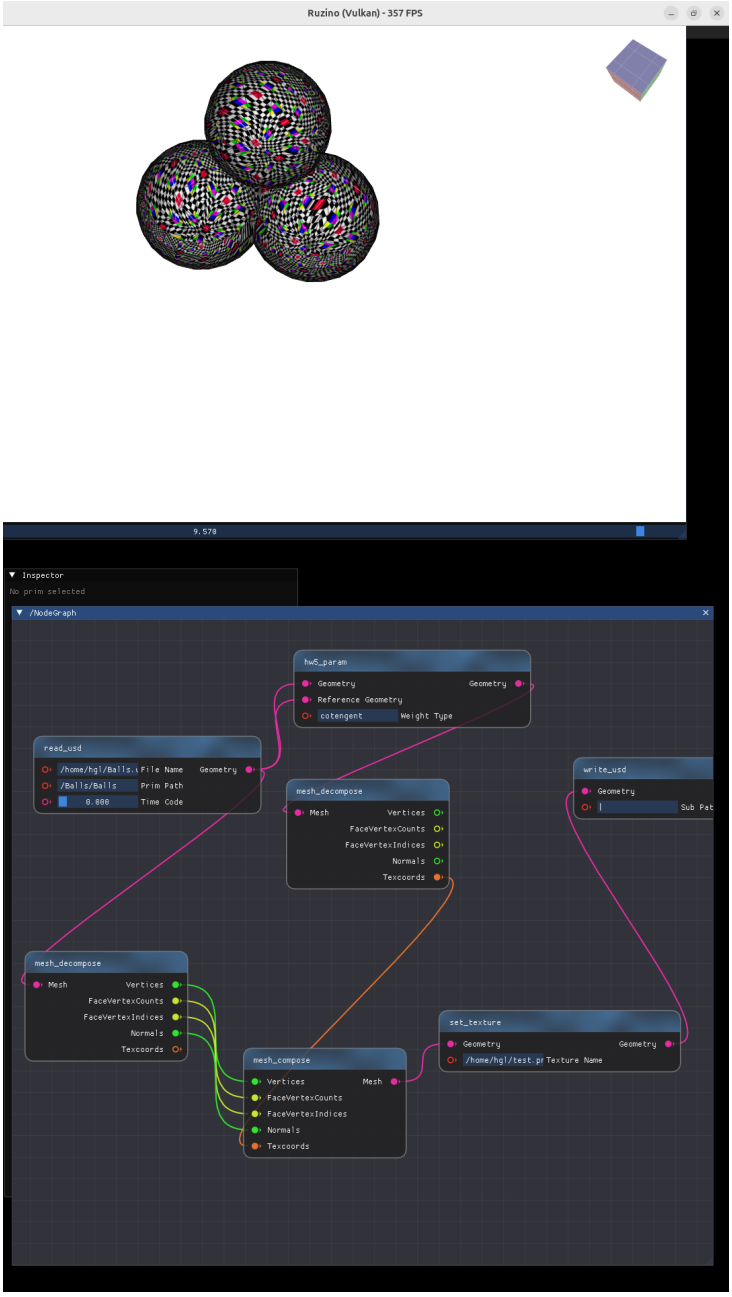

图 6: 贴图在立体视图下的最终效果（高分辨率视图）

5 结果文件与提交说明

- 已在本地生成并保存 USD：`Homeworks/5_tutte_parametrzation/pdf/Results.usd`（请确认该文件在你打包提交时包含在压缩包中）。- 提交包建议包含：修改过的节点源文件、`report.pdf`（由本 `report.tex` 编译）、`stage.usdc` 或 `Results.usd`、`data/` 下的测试模型与纹理。

6 讨论与结论

- Uniform 权重结果简单稳定，但在纹理分布上通常会产生非均匀拉伸。Cotangent 权重考虑了原始几何角度，能在保形或减少局部失真方面更优。- 将参数化 UV 回写到原始立体，并通过 `set_texture` 赋材质，是验证参数化质量最直观的方法。- 如需进一步量化，可计算拉伸度量或角度失真并绘制热图（可扩展为下一步工作）。